Introduction

Most Python tutorials teach you in isolation. One lesson on classes. One lesson on testing. One lesson on async. You finish each one feeling capable, and then you sit down to build something real and nothing connects. The classes don't know about the tests. The tests don't know about the async layer. And nobody ever gets to the part where you type `pip install .` and it just works.

That gap is exactly what shows up in technical interviews and on real engineering teams. Interviewers don't just want to know if you can write a `for` loop — they want to see if you can structure a small system: model the domain cleanly, protect it with tests, make it fast where it needs to be, and hand it off in a form someone else can actually run. This lesson walks through building one small application — a command-line expense tracker — that does all four, so you can see how the pieces actually fit together instead of living as four separate skills you've never combined.

Problem Overview

Here's the goal, striped down to its core: build a command-line expense tracker that a stranger could install and run without ever seeing your project folder.

That means:

- A clean domain model for expenses and a ledger of them.
- Automated tests that catch regressions before a user does.
- Live currency conversion fetched concurrently instead of one request at a time.
- A packaging setup so the whole thing installs as a real command, not a script you run by remembering its file path.

None of this requires new syntax. It requires knowing how dataclasses, pytest, `asyncio`, and `pyproject.toml` sit next to each other in one working project.

Example

Imagine adding two expenses to the ledger:

```
Coffee     — 4.50 USD
Groceries  — 12.00 USD
```

Calling `ledger.total()` should return `16.50`. That's the obvious case. But two more examples matter just as much:

- An **empty ledger** — `total()` should return `0`, not raise an error.
- A ledger with expenses in **different currencies** — the total only makes sense once every amount is converted to one currency, which is exactly where the async layer comes in.

Each of these is a separate test case later, because each one fails in a different way if the code is wrong.

Intuition

An experienced engineer doesn't start with tests, async, or packaging. They start with the plainest possible objects that describe the problem, because everything downstream — tests, concurrency, distribution — is easier to reason about when the core domain is simple and honest.

So the first question is: what are the *nouns* in this problem? An expense (a description, an amount, a currency) and a ledger (a collection of expenses that can be totaled). Nothing about HTTP requests or currency rates belongs in these classes yet — that's a separate concern that gets layered on later, which is itself a lesson: keep the thing that models your data separate from the thing that fetches data for it.

Once that core exists, the next question an engineer asks isn't "does it work" — it's "how do I know if I break it later?" That's what pushes you toward tests before you touch anything else. Only after the model is trustworthy does it make sense to make it fast (async) and shippable (packaging), because optimizing or distributing something you don't trust just multiplies the blast radius of a bug.

Brute Force Solution

Idea: Skip structure entirely — track expenses as raw tuples or dictionaries in a script, fetch currency rates one at a time with `requests`, and run everything as `python main.py`.

Algorithm: 1. Store expenses as `list[dict]`. 2. Loop over them to compute a total. 3. For each unique currency, make a blocking HTTP call to fetch its rate. 4. Print the result.

Advantages: Fast to write for a one-off script; no dependencies beyond `requests`.

Disadvantages: No safety net — a typo in a dict key fails silently. No way to distribute it — anyone who wants to run it needs your folder and the right command memorized. Network calls happen sequentially, so twenty expenses in twenty currencies means twenty round trips stacked end to end.

Complexity: Computing the total is O(n) for n expenses. Fetching rates sequentially is O(k × latency) for k unique currencies — that latency term is the whole problem, because it scales with network round-trip time, not CPU work.

Optimal Solution

The optimal version isn't algorithmically different — it's *structurally* different, and structure is what makes the same logic testable, fast, and distributable.

Step 1 — Model the domain with dataclasses.

`Expense` and `Ledger` are declared as `@dataclass` instead of hand-written classes. A dataclass generates `__init__` and `__repr__` for you, so the class body only shows the fields and the methods that actually matter — the boilerplate that would otherwise bury the logic is gone.

Step 2 — Lock behavior in with pytest before adding anything else.

Three tests, each covering a distinct case:

Test	What it checks	Why it matters
Single expense	Total equals that one amount	Baseline sanity check
Multiple expenses	Total equals their sum	The case most people remember
Empty ledger	Total equals zero, no crash	The edge case people skip — and the one that breaks silently when `total()` is refactored later

Step 3 — Fetch currency rates concurrently with asyncio.

`fetch_rate` is a single coroutine that awaits one network call for one currency. `attach_rates` first de-duplicates which currencies actually appear in the ledger — no reason to fetch USD twice — then hands all the resulting coroutines to `asyncio.gather`, which launches them together and waits for all of them at once instead of one after another.

```
Sequential:  [req1]→[req2]→[req3]→...→[req20]   (20x latency)
Concurrent:  [req1]
             [req2]
             [req3]                              (~1x latency)
             ...
             [req20]
```

Step 4 — Package it so it installs like real software.

A `pyproject.toml` names the package, lists `aiohttp` as the one dependency, and — the part that actually matters — defines an entry point mapping the command `expenses` to the `main` function in `expenses.cli`. `pip install .` reads that entry point and writes an `expenses` executable into your environment's `bin` folder. After that, `expenses` is a command, not a script whose path you have to remember.

Python Code

```
# expenses/models.py
from dataclasses import dataclass, field


@dataclass
class Expense:
    description: str
    amount: float
    currency: str = "USD"


@dataclass
class Ledger:
    expenses: list[Expense] = field(default_factory=list)

    def add(self, expense: Expense) -> None:
        self.expenses.append(expense)

    def total(self) -> float:
        return sum(e.amount for e in self.expenses)

    def currencies(self) -> set[str]:
        return {e.currency for e in self.expenses}
```

```
# expenses/rates.py
import asyncio
import aiohttp

RATE_URL = "https://api.exchangerate.host/latest?base={currency}"


async def fetch_rate(session: aiohttp.ClientSession, currency: str) -> tuple[str, float]:
    async with session.get(RATE_URL.format(currency=currency)) as resp:
        data = await resp.json()
        return currency, data["rates"]["USD"]


async def attach_rates(ledger) -> dict[str, float]:
    currencies = ledger.currencies()
```

```
    async with aiohttp.ClientSession() as session:
        results = await asyncio.gather(
            *(fetch_rate(session, c) for c in currencies)
        )
    return dict(results)
```

```
# expenses/cli.py
import asyncio
from expenses.models import Expense, Ledger
from expenses.rates import attach_rates


def main() -> None:
    ledger = Ledger()
    ledger.add(Expense("Coffee", 4.50))
    ledger.add(Expense("Groceries", 12.00))
    print(f"Total: {ledger.total():.2f}")


if __name__ == "__main__":
    main()
```

```
# pyproject.toml
[project]
name = "expenses"
version = "0.1.0"
dependencies = ["aiohttp"]

[project.scripts]
expenses = "expenses.cli:main"

[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"
```

Code Walkthrough

- `@dataclass` on `Expense` and `Ledger` generates `__init__`, `__repr__`, and `__eq__`, so the class body is just fields and real methods.
- `field(default_factory=list)` avoids the classic mutable-default-argument bug — a bare `= []` would share one list across every `Ledger` instance.
- `Ledger.total()` uses a generator expression inside `sum()`, so an empty list correctly sums to `0` without a special case.
- `fetch_rate` is a coroutine — calling it doesn't run it, it returns an awaitable that `asyncio.gather` schedules.

- `attach_rates` de-duplicates currencies with a set comprehension before firing off requests, so fetching the same currency twice never happens.
- `asyncio.gather(*coroutines)` unpacks the coroutines as separate arguments and runs them concurrently on the event loop, returning results in the same order they were passed in.
- The `[project.scripts]` table in `pyproject.toml` is what turns `expenses.cli:main` into a real `expenses` command after install.

Dry Run

Starting state: empty ledger.

1. `ledger.add(Expense("Coffee", 4.50))` → `ledger.expenses = [Expense("Coffee", 4.50, "USD")]`
2. `ledger.add(Expense("Groceries", 12.00))` → `ledger.expenses` now has two entries.
3. `ledger.total()` sums `4.50 + 12.00` → `16.50`.
4. Output: `Total: 16.50`.

Every step is backed by a test: the single-add test covers step 1's shape, the multi-add test covers step 3's arithmetic, and the empty-ledger test guarantees step 3 doesn't crash before any `add` call happens.

Complexity Analysis

- **Time:** `Ledger.total()` is O(n) for n expenses — one pass, no way around touching every entry once. Fetching rates concurrently is bounded by the *slowest single request*, not the sum of all of them, so it's O(max latency) instead of O(k × latency) for k currencies.
- **Space:** O(n) for storing n expenses, O(k) for k unique currencies' rates — both linear and both necessary, since you can't total or convert what you haven't stored.
- These bounds are correct because nothing in the design re-scans the ledger more than once per operation, and the concurrency doesn't change how much data you hold, only how long you wait for it.

Alternative Solutions

- **Threading instead of asyncio:** Works for I/O-bound fetches too, but adds thread-safety concerns for shared state (the ledger) that `asyncio`'s single-threaded event loop sidesteps entirely. For a small number of network calls, `asyncio` is simpler and just as fast.

- **A database instead of in-memory dataclasses:** Reasonable once the tracker needs to persist across runs, but premature here — it would add a schema and a migration story to a project whose actual bottleneck is structure and shipping, not storage.

Edge Cases

- **Empty ledger** — `total()` must return `0`, not raise.
- **Duplicate currencies** — `attach_rates` must fetch each one exactly once, not once per expense.
- **Negative or zero amounts** — a refund or correction entry should still sum correctly.
- **A single failing rate request** — `asyncio.gather` raises the first exception it hits and cancels the rest by default, which silently kills an otherwise-fine batch unless you pass `return_exceptions=True` and handle failures per-currency.
- **Renaming the entry point after install** — `expenses` keeps running the *old* function until you `pip install .` again, because the wrapper script was written at install time, not resolved dynamically.

Common Mistakes

1. Using a mutable default argument (`expenses: list = []`) instead of `field(default_factory=list)`, causing every `Ledger` to share one list.
2. Writing tests only for the happy path and skipping the empty-input case — exactly the one that breaks silently later.
3. Assuming `asyncio.gather` skips failed requests by default — it doesn't, it cancels the whole batch on the first exception.
4. Forgetting to de-duplicate currencies before fetching, wasting requests on the same rate.
5. Editing `pyproject.toml`'s entry point after installing and expecting the change to take effect without reinstalling.
6. Mixing async and sync code carelessly — calling a coroutine without `await` and getting a coroutine object instead of a result.

Interview Questions

- Why use `@dataclass` instead of a plain class here, and when would you *not* use one?
- What does `asyncio.gather` do differently from running the same coroutines in a loop with `await`?

- How would you make `gather` continue past a single failed request instead of cancelling everything?
- Why test the empty-ledger case specifically, and what kind of bug does it catch that the two-item test wouldn't?
- What actually happens when you run `pip install .` — what does the entry point mechanism do under the hood?

Similar Problems

- **Design a Rate Limiter** — another problem where correctness depends on concurrent behavior, not just sequential logic.
- **Design an LRU Cache** — same theme of building a small, self-contained data structure with clear invariants worth testing.
- **Merge Intervals** — practicing the habit of identifying the edge case (empty input, single interval) before trusting a solution.

Key Takeaways

Classes gave the project structure. Tests gave it a safety net. Async gave it speed. Packaging gave it to the world. None of these are separate skills in a real project — they're four angles on the same small system, and the moment you connect them, you've built something a stranger could actually run, not just a snippet that works on your machine.

Watch the Video

For the full walkthrough with live coding and the async gather question worked out on screen, watch the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series built around one idea: tutorials teach pieces, but real software is what happens when those pieces are forced to work together. Each lesson takes a concept most developers already know in isolation and shows where it actually lives inside a working project.

Call To Action

If this connected some dots for you, drop a like on the video — that's genuinely how the channel knows what's landing. Subscribe on YouTube for the rest of the series, and subscribe to the Substack newsletter for the write-ups that go deeper than the video has time for. Comment with your guess on what `asyncio.gather` does when one request fails, and share this with anyone still writing classes, tests, and async code that never talk to each other.

Quick Review

Why build a capstone before starting a new topic?

Forces integrating OOP, tests, async, and packaging together—skills only click when combined, not learned in isolation.

Why write pytest tests before adding async fetching?

Locks in current behavior as a safety net, so later concurrency changes can't silently break existing logic.

Sync code vs async code: when does async actually help?

Async wins when waiting on I/O (network/disk); it doesn't speed up pure CPU-bound work.

Trickiest bug when adding async to an existing sync codebase?

Mixing blocking calls inside async functions, which silently stalls the whole event loop instead of raising an error.

What problem does packaging (pyproject.toml, entry points) solve?

Makes the project pip-installable and runnable elsewhere, instead of only working from one developer's folder.

Interview follow-up: why not just add print statements instead of tests?

Prints verify one manual run; tests re-verify automatically on every future change, catching regressions early.

Python Lesson 37: Capstone: Building a Complete Application (Combining OOP, testing, async or concurrency, and packaging into a real shipped project) — free Python cheat sheet